

JSON Conversion Service

Requirements Document — Intern Assignment

1. Purpose

Interns will build a standalone JSON-to-JSON conversion service that mimics, in simplified form, the “custom format translator” logic used in production. The service takes a raw extraction-result JSON (Titan-style) as input, converts its fields array into a business-mapped custom format using MongoDB-stored field mapping configs, keeps a small fixed set of metadata fields as pass-through, and persists the result into MongoDB.

This is a learning exercise focused on config-driven field mapping and JSON transformation using a simple Flask/FastAPI + MongoDB service — not a re-implementation of the full production wrapper (no queues, no LLMs, no business-rule engine, no S3).

2. Scope

In scope

- REST API that accepts the raw input JSON and returns a converted JSON.
- Business-type detection (AUTO vs MORTGAGE) per document.
- Config-driven field name mapping fetched from MongoDB (per business type).
- Building the output document(s) and fields array, including dropdown-value unwrapping and Object List fan-out into multiple output documents (see Section 6).
- Passing through a fixed, limited set of metadata fields (Section 4).
- Inserting the converted document(s) into a MongoDB output collection.
- Returning a success/failure response.

Out of scope (do not build)

- RabbitMQ / async queues, S3 uploads, callbacks.
- LLM calls, business rules engine, NTP / date-smart comparisons.
- Value-level “smart” transformations (address comma-stripping, VIN correction, etc.) — optional stretch goal only, see Section 10.
- Auth / security beyond basic input validation.
- Alternate-candidate selection, word-coordinate export, split-field re-merging — see Appendix A.

3. Tech Stack

Layer	Choice
Language	Python 3.10+
Web framework	Flask or FastAPI (either is acceptable)
Database	MongoDB (local instance)
Driver	pymongo (Flask) or motor / pymongo (FastAPI)
Config storage	MongoDB collection (see Section 5.1)

4. Fixed Output Metadata Fields

Every converted document in the output must contain only the fields below, plus the fields array. Do not add any other top-level metadata fields, even if present in production examples.

Output field	Type	Source / Derivation rule
upload_request_id	string	requestId (top level) — direct copy
upload_request_status	string	status (top level) — direct copy
document_id	string	documents[i].id — direct copy
document_name	string	documents[i].name
document_file_type	string	documents[i].fileType
document_processing_status	string	documents[i].status
file_uploaded_timestamp	{"\$date": ISO8601}	documents[i].lastModifiedDate, parsed from "YYYY-MM-DD HH:mm:ss:SSS" and wrapped
document_received_timestamp	{"\$date": ISO8601}	documents[i].documentReceivedDate, same parse/wrap rule
document_extraction_start_timestamp	{"\$date": ISO8601}	documents[i].documentExtractionStartDate, same parse/wrap rule
business_type	string (AUTO / MORTGAGE)	documents[i].docTypeHierarchy[] where classificationName == "CollateralType" → its value, uppercased
document_type	string	documents[i].docTypeHierarchy[] where classificationName == "DocumentType" → its value, uppercased
document_page_count	int	len(documents[i].pages)
original_file_page_count	int	Constant 0 (not derivable from input)
original_file_blank_pages	int	documents[i].totalBlankPages, default 0 if missing
list_document	bool	true if docTypeHierarchy[] has classificationName "LIST_TYPE" with value "LIST"/"LIIST", else false
optional_parameters	object	documents[i].optionalParams, passed through as-is
source_document_url	string	documents[i].sourceDocumentUrl
fields	array	See Section 6

account_id cannot be derived from the raw JSON — it must come from a config lookup. This is intentional: it teaches combining external config with request data.

IMPORTANT: a single input document can produce MORE THAN ONE output document — see Section 6.1. Every field above (and every non-Object-List field in the fields array) is identical across all output documents generated from the same input document; only document_name's group_index and the Object-List-derived entries in the fields array change between them.

5. MongoDB Collections

5.1 Field Mapping Config — field_transformation_config

One document per business type. Seed this collection with the exact mapping arrays supplied (autoFieldTransformationConfig / mortgageFieldTransformationConfig).

```
{
  "business_type": "AUTO",
  "mappings": [
    { "titanFieldName": "Policy Number", "customFormatFieldName": "policy_number",
      "fieldType": "Single Value", "fieldDataType": "String" },
    { "titanFieldName": "* Collateral Year", "customFormatFieldName": "collateral_year",
      "fieldType": "Single Value", "fieldDataType": "String" }
```

```
]
}
```

5.3 Output Collection — custom_format_translated_documents

Stores the final converted document(s). As explained in Section 6.1, one input document can produce several stored documents here — one per Object List group (e.g. one per vehicle) — not one per input document. Each stored document follows the exact shape described in Sections 4 and 6.

6. Fields Array Conversion Logic

6.1 One input document can produce MULTIPLE output documents

`documents[i].fields` normally contains at most one Object List field (named "Collateral" in the sample data — one entry per vehicle). The number of output documents generated for that input document equals the number of groups in that field's values array. If the document has no Object List field, exactly one output document is generated.

Rule:

- `number_of_output_documents` = `len(object_list_field.values)` if an Object List field is present, else 1.
- Every non-Object-List field (mapped via config) is identical and repeated across all output documents generated for this input document.
- For the output document at group index `i` (0-based), only the dicts belonging to `values[i]` are converted into fields entries (with `titan_field_name` / `titan_field_id` / `titan_object_id`); dicts from every other group are excluded from that output document.
- `document_id` is identical across all output documents generated from the same input document; only `document_name`'s trailing `group_index` (1-based: `i + 1`) differs.

Worked example — a document whose Collateral field has 2 groups (2 vehicles) produces 2 output documents:

```
document_name: "..._1.pdf"      document_name: "..._2.pdf"
fields: [                      fields: [
  ...common fields (policy_number,  ...common fields (policy_number,
    policy_holder_name, carrier_name,  policy_holder_name, carrier_name,
    etc.) — SAME in both documents    etc.) — SAME in both documents
  { field_name: collateral_make,      { field_name: collateral_make,
    field_value: "FORD", ... },      field_value: "TOYO", ... },
  ... rest of group[0]'s fields      ... rest of group[1]'s fields
]
```

Assumption: this data model has at most one Object List field per input document. If none is present, `group_index` defaults to 1 and exactly one output document is produced with an empty set of `titan_*` fields.

6.2 Shape A — Simple / scalar field

value is a plain string/number/bool.

```
{ "name": "Policy Number", "type": "Single Value", "dataType": "string", "value":
  "170200044270" }
```

maps to:

```
{ "field_name": "policy_number", "field_value": "170200044270",
  "field_type": "Single Value", "field_data_type": "String" }
```

6.3 Shape B — Dropdown field (value is a nested object)

When a field's `dataType` is "dropdown" (case-insensitive), its value is not a plain scalar — it is a value object (dict) that must be unwrapped. This mirrors how production resolves dropdown fields.

```
{ "name": "Comprehensive Coverage Flag", "type": "Single Value", "dataType": "dropdown",
  "valueObject": { "value": "Yes", "dropdownValue": "Yes" } }
```

Extraction rule: read the `dropdownValue` key from the nested `valueObject`, and trim it. That trimmed string becomes `field_value`.

```
def resolve_value(field):
    if field["dataType"].lower() == "dropdown":
        value_obj = field["valueObject"]      # dict, not a scalar
```

```

    return value_obj.get("dropdownValue", "").strip()
    return field["value"]

```

Reference: production implements this in ResponseFieldUtils.java (getDropdownValue / parseDropdownValue) — it reads the "dropdownValue" key out of the field's value object and trims it. See Appendix B.

Out of scope: production sometimes maps the unwrapped dropdownValue further through a business rule (e.g. an "Insurance Type" dropdown with dropdownValue "Continuous" becomes output value "C" via a code lookup table). This assignment's resolve_value() should return the dropdownValue text unchanged — do not attempt to replicate that lookup table.

6.4 Shape C — Object List field (e.g. "Collateral") — one group per output document

An Object List field's values key is a fixed, two-level nested structure: a list of groups, where each group is a list of dicts. Each group corresponds to ONE output document (Section 6.1) — it is not merged into a single document's fields array. Each dict inside a group represents one sub-field belonging to that group (e.g. one vehicle) and becomes its own entry in that output document's fields array.

```

"values": [
  [
    { "fieldName": "* Collateral Make", "value": "FORD", "fieldId": "c3adf...",
      "objectId": "6ce8...", "fieldType": "Single Value", "dataType": "string" },
    { "fieldName": "* Collateral Model", "value": "F150", "fieldId": "8beb...",
      "objectId": "6ce8...", "fieldType": "Single Value", "dataType": "string" }
  ],
  [
    { "fieldName": "* Collateral Make", "value": "FORD", "objectId": "7df0...", ... },
    { "fieldName": "* Collateral Model", "value": "BRONCO SPORT", "objectId": "7df0...", ... }
  ]
]

```

Each mapped dict becomes:

```

{
  "field_name": "collateral_make", "field_value": "FORD",
  "field_type": "Single Value", "field_data_type": "String",
  "titan_field_name": "* Collateral Make", "titan_field_id": "c3adf...",
  "titan_object_id": "6ce8..."
}

```

Nesting depth is fixed at exactly this level — field.values → list of groups → list of dicts — confirmed against production (CustomDocumentFormatGeneratorService.java + ResponseFieldUtils.convertSubExtractionsToResponseField). Do NOT implement recursion for further nested Object Lists inside a dict; it does not occur in this data model. objectId is shared by every dict within the same group — if an individual dict is missing it, fall back to the first dict's objectId in that group.

A dict inside a group can itself use Shape B (dropdown) for its value — apply the same resolve_value() unwrapping rule from Section 6.3 before mapping.

6.5 Full conversion algorithm

Produces a LIST of output documents per input document (length 1, unless an Object List field with multiple groups is present):

```

def convert_document(document, business_type):
    mapping = fetch_mapping_config(business_type)      # field_transformation_config
    mapping_lookup = { m.titanFieldName: m for m in mapping.mappings }

    # At most one Object List field per document (Section 6.1)
    object_list_field = next((f for f in document["fields"] if f["type"] == "Object List"), None)
    groups = object_list_field["values"] if object_list_field else [[]]  # 1 empty group if none

    # Fields that are NOT the Object List — identical across every output document
    common_fields = []
    for field in document["fields"]:
        if field is object_list_field:
            continue
        m = mapping_lookup.get(field["name"])
        if not m:
            continue  # unmapped -> drop
        common_fields.append({
            "field_name": m.customFormatFieldName,

```

```

        "field_value": resolve_value(field),
        "field_type": field["type"],
        "field_data_type": capitalize(field["dataType"]),
    })

output_documents = []
for group_index, group in enumerate(groups):
    group_fields = []
    shared_object_id = group[0].get("objectId") if group else None
    for subfield in group:
        m = mapping_lookup.get(subfield["fieldName"])
        if not m:
            continue # unmapped -> drop
        group_fields.append({
            "field_name": m.customFormatFieldName,
            "field_value": resolve_value(subfield),
            "field_type": subfield["fieldType"],
            "field_data_type": capitalize(subfield["dataType"]),
            "titan_field_name": subfield["fieldName"],
            "titan_field_id": subfield.get("fieldId"),
            "titan_object_id": subfield.get("objectId", shared_object_id),
        })

    output_documents.append({
        **build_metadata(document, group_index), # Section 4; document_name uses
        "fields": common_fields + group_fields,
    })

return output_documents # one input document -> len(groups) output documents

```

`capitalize(dataType): "string" → "String"` (only value seen in samples — keep this a simple literal mapping, not general title-casing, in case new data types appear).

Unmapped fields (no entry in `mapping_lookup`) are silently dropped from the output, but log at INFO level with the titan field name so it stays traceable.

7. API Contract

POST /api/convert

Request body: the raw extraction JSON, in the shape shown above (top-level `requestId`, `status`, `documents[]`). Assume it is already valid JSON, not a doubly-escaped string, to keep the assignment simple.

Response (success):

```

{
  "status": "SUCCESS",
  "upload_request_id": "523cb210-345b-40be-ba56-64fe9ef10148",
  "documents_converted": 2,
  "document_ids": ["38d399c0-...", "38d399c0-..."]
}

```

documents_converted counts OUTPUT documents actually inserted, after the Section 6.1 fan-out — not the number of entries in the input documents[] array. A single input document with a 2-group Object List field contributes 2 to this count (and its id appears twice in document_ids, once per output document).

Response (failure):

```

{ "status": "FAILURE", "error": "<message>" }

```

HTTP 400 for malformed/missing input, HTTP 500 for unexpected/internal errors.

8. Validation Requirements

- Reject requests missing `requestId`, `status`, or `documents`.

- If a document's business type can't be resolved (no CollateralType in docTypeHierarchy), skip that document, log a warning, and continue processing the rest — don't fail the whole request.
- If the mapping config collection has no entry for a resolved business type, fail that document's conversion with a clear error message (don't silently produce an empty fields array).
- If account_config has no entry for the client_id, set account_id to null and log a warning (don't crash the request).
- If a dropdown field's value object is missing the dropdownValue key, set field_value to "" and log a warning.

9. Testing Requirements

Unit tests (pytest)

- Business type detection (AUTO / MORTGAGE / missing classification).
- Simple field mapping (mapped field kept, unmapped field dropped).
- Dropdown field unwrapping (dropdownValue present, missing, and non-dropdown fields unaffected).
- Object List fan-out (a field with N groups produces N output documents, not N sets of flattened entries merged into one document; common fields repeat identically in every output document; correct titan_* passthrough and shared objectId fallback within each group).
- Zero-group edge case: a document with no Object List field produces exactly 1 output document.
- Metadata derivation (document_name construction, model_id URL parsing, timestamp parsing/wrapping).
- Account lookup (found / not found).
- Mapping config lookup (found in AUTO but not MORTGAGE and vice versa).

Integration test

- Full POST /api/convert call against a test Mongo instance (or mongomock), asserting the inserted document in custom_format_translated_documents matches the expected shape.

10. Stretch Goals (optional, not required for MVP)

- Value-level cleanup transformations (e.g. stripping commas from concatenated addresses).
- In-memory caching of mapping configs to avoid a Mongo round-trip per request.
- Handling isCorrected / correctedValueObject precedence for dropdown fields, mirroring production (see Appendix B).

11. Deliverables & Acceptance Criteria

- Working Flask/FastAPI service with POST /api/convert.
- MongoDB seed script for field_transformation_config (AUTO + MORTGAGE) and account_config.
- Given the sample input (one input document whose Collateral field has 2 groups), the service produces 2 documents in custom_format_translated_documents — sharing the same document_id, differing only in document_name's group_index and their group-specific Collateral fields — matching the field mapping and metadata rules above.
- Unit + integration tests passing.
- README with setup steps (Mongo connection string, how to seed config, how to run, example curl request).

Appendix A — Other field shapes present in production (informational only)

These exist in the real Titan extraction format and are called out for awareness. They are out of scope for this assignment — do not implement them unless attempting a stretch goal.

Shape	What it is	Where it lives in production
Alternate candidates	Each field can carry alternateCandidates: other extractor outputs (GENIE/BIBOT/COGNI) with their own value/confidence, used by business rules to pick a winner.	ProcessDocument.java (field.getAlternateCandidates())
Word coordinates	Each field can carry wordCoordinates mapping the value back to OCR token positions on the page image. Not exported to the final custom JSON.	ResponseFieldUtils.java (field.getWordCoordinates())

Shape	What it is	Where it lives in production
Split fields	Some output fields (e.g. an address) are built by merging several separately-extracted sub-fields (House Number, Street 1, Street 2, etc.) at output time.	SplitFieldsMergeConfig.java
Confidence	Every field carries a confidence score, sometimes overridden by business rules.	Various rule classes

Appendix B — Reference to production implementation

For interns curious how the real wrapper does this (optional reading, not required):

- Dropdown unwrapping: ResponseFieldUtils.java — getDropdownValue() / parseDropdownValue() / getResponseFieldValue(). Reads the "dropdownValue" key from the field's value object (or correctedValueObject if the field was corrected), trims it.
- Object List flattening: CustomDocumentFormatGeneratorService.java, using ResponseFieldUtils.convertSubExtractionsToResponseField() and convertFieldsToTranslatedDocumentFields(). Iterates each group in field.getValues(), converts each dict to a field, and extracts the shared objectId from the first dict via extractObjectIdFromCollateralItem().

Class/method names above are Java-specific and do not need to be replicated — only the JSON shapes and rules they encode matter for this Python assignment.